

Техническая сводка по проделанной работе (для НИР)

****Проект:**** TreeD MainshellIOS

****Ветка/срез:**** `dev`

****Дата среза:**** 12.02.2026

****Тип документа:**** промежуточная техсводка средней детализации

****Источник фактов:**** репозиторий (код, конфиги, документация, история коммитов)

1. Что уже сделано в проекте

На текущем этапе сформирован рабочий контур развертывания и эксплуатации системы управления принтером на базе связки ****Raspberry Pi + MKS Robin Nano V1.2 (RN12)**** с прошивкой ****Klipper****.

Ключевой результат — проект приведен к воспроизводимой модели: единый вход через `loader/loader.sh`, детерминированный порядок шагов, разделение source-of-truth и runtime-слоя, а также формализованные проверки после деплоя.

В рамках работ закрыты базовые вопросы:

- ? подготовка и обновление прошивки RN12 под Klipper;
- ? переход и поддержка двух транспортов MCU (`usb`/`uart`) с приоритетом `uart`;
- ? фиксация рабочего профиля `rn12\_hbot\_v1` в модульной структуре;
- ? интеграция Moonraker-компонента для вызовов shell-команд из макросов;
- ? стандартизация verify-проверок по MCU, сервисам, теме и камере;
- ? повышение эксплуатационной устойчивости (fail-fast для обязательных шагов, best-effort для опциональных).

1.1. Ключевые пробелы (критично для НИР/диплома)

На текущем этапе остаются два принципиальных пробела, которые должны быть формально закрыты в материалах НИР:

1. Не зафиксирована как source-of-truth полная аппаратная часть:

- BOM-ведомость;
- конкретные модели драйверов шаговых двигателей;
- детальная схема питания и защит.

2. Не собраны полевые метрики эксплуатации:

- надежность по длительным прогонам;
- частота и типология ошибок;
- статистика по длительным тестам печати.

2. Результаты по прошивке и MCU-контур

2.1. Зафиксирован целевой контур прошивки RN12

Для платы ****MKS Robin Nano V1.2**** описан и закреплён полный цикл:

- ? сборка Klipper под ****STM32F103****;
- ? параметры сборки: `28KiB bootloader`, `8 MHz crystal`, `Serial on USART1 PA10/PA9`, `baud 250000`;
- ? подготовка бинарника для загрузчика Robin Nano через `update\_mks\_robin.py`;
- ? эталонный артефакт хранится в репозитории: `firmware/rn12/ROBIN\_NANO.bin`.

Таким образом, прошивка не является «одноразовой ручной процедурой», а оформлена как повторяемый процесс с документированными шагами и контролем результата (признак успешной прошивки: `ROBIN\_NANO.CUR` на microSD).

2.2. Реализован двухтранспортный контур MCU (USB/UART)

В проекте поддерживаются два режима связи хост↔MCU:

- ? `usb` (через `/dev/serial/by-id/\*`);

? `uart` (через `TREED\_MCU\_UART\_DEV`, по умолчанию `/dev/serial0`).

Критичное решение: для RN12 принят ****двухфазный переход на UART****:

1. подготовка Raspberry Pi (UART-контур, отключение конфликтов serial-console, reboot);
2. переключение runtime-профиля на UART после подтверждения физической коммутации.

Это снижает риск потери связи с MCU при миграции с USB на UART и исключает смешение шагов «подготовка ОС» и «переключение рабочего serial».

2.3. Автоматизирована подстановка `serial` в профиль

Шаг `loader/steps/klipper-profiles.sh`:

- ? валидирует выбранный транспорт;
- ? проверяет корректность и доступность устройства;
- ? автоматически обновляет `serial:` в `mcu\_rn12.cfg`;
- ? в случае неоднозначности/ошибки завершает шаг с ошибкой (без silent-skip).

Это устраняет типовой класс ошибок, когда конфиг формально применен, но фактически ссылается на неверный serial-путь.

3. Выбор электронных компонентов и аппаратных параметров (зафиксированная конфигурация)

Ниже перечислены решения, которые подтверждаются текущими конфигами и документацией проекта.

3.1. Базовые элементы контура управления

- ? ****MCU-плата:**** MKS Robin Nano V1.2 (RN12).
- ? ****Host:**** Raspberry Pi с MainsailOS/Klipper/Moonraker.
- ? ****Транспорт MCU:**** USB и UART 3.3V TTL (приоритетный рабочий режим — UART).
- ? ****Носитель для MCU-flash:**** microSD (FAT32) с `ROBIN\_NANO.bin`.

3.2. Фиксированная коммутация UART

Для RN12 закреплено подключение через WiFi-UART header (USART1):

- ? RN12 `PA9 (TX)` -> Pi `GPIO15 / pin 10 (RX)`;
- ? RN12 `PA10 (RX)` -> Pi `GPIO14 / pin 8 (TX)`;
- ? `GND` -> `GND`.

Отдельно зафиксировано, что в данном контуре используется именно `PA9/PA10`, а не `PB10/PB11`.

3.3. Конфигурация исполнительных и измерительных цепей (по профилю)

В профиле `rn12\_hbot\_v1` зафиксированы:

- ? оси X/Y/Z, пины шаг/направление/enable и концевики;
- ? экструдер с ограничениями подачи и `pressure\_advance`;
- ? термисторы ****NTC100K_B3950**** для хотэнда и стола;
- ? стол с PID-регулированием и `verify\_heater`;
- ? вентилятор модели `[fan]`;
- ? бипер на `PC5` с макросами `M300/BEEP` и стартовым сценарным сигналом.

Принципиально: параметры разнесены модульно по ответственности, без «монолитного» `printer.cfg`.

3.4. Ограничение текущей фиксации по электронике

В данном срезе репозитория ****не зафиксированы как source-of-truth****:

- ? конкретные модели драйверов шаговых двигателей;
- ? детальная BOM-ведомость;
- ? схема питания (БП, защита, распределение линий) в формате отдельного инженерного документа.

Эти разделы рекомендуется добавить отдельным пакетом к НИР (см. раздел 8).

4. Результаты по коммутации, слоям конфигов и runtime-интеграции

4.1. Внедрена модель слоев и ownership

Сформирована устойчивая модель:

- ? репозиторий — канонический source-of-truth;
- ? staging (`/home/pi/treed/klipper`) — промежуточный слой;
- ? runtime (`/home/pi/printer\_data/config`) — примененный рабочий слой.

Это уменьшает риск дрейфа конфигураций между Git и устройством и облегчает воспроизводимость на новых платах.

4.2. Стабилизирована include-цепочка профиля Klipper

Точка входа: `klipper/printer.cfg`, далее модульные include:

`mcu`, `printer\_base`, `steppers`, `extruder`, `bed\_heater`, `fans`, `beeper`, `macros`, `ui`, `local\_overrides`.

Локальные runtime-overrides (`local\_overrides.cfg` и др.) сохраняются при деплое, что позволяет развивать репозиторийный слой без потери локальных настроек устройства.

4.3. Интеграция Moonraker + runtime-скриптов

Добавлен компонент `moonraker/components/treed\_shell\_command.py`, обеспечивающий вызовы shell-команд из макросов Klipper через remote-method `machine.shell\_command`.

Практический результат:

- ? макросы печати и камеры связаны с runtime-скриптами (`treed\_cam\_\*`);
- ? сервисный контур (в т.ч. beeper boot-once) может запускаться из безопасной конфигурационной точки Moonraker.

5. Ключевые инженерные решения и их эффект

5.1. Fail-fast для обязательных шагов

В `loader.sh` обязательные шаги выполняются в строгом режиме: отсутствие скрипта или ошибка шага прерывает прогон.

Эффект: исключены «тихие» полуробочие состояния после установки.

5.2. Best-effort для опциональных подсистем

Камера (`crowsnest`) и часть шага KlipperScreen допускают optional-поведение, если не нарушается базовый контур печати.

Эффект: отсутствие опциональной периферии не блокирует базовый provisioning.

5.3. Усиленный verify-контур

Шаг `verify` проверяет:

- ? свежие ошибки MCU в журнале;
- ? соответствие runtime `serial` выбранному транспорту;
- ? доступность UART-девайса и права доступа;
- ? статус ключевых сервисов и часть UI/камерного контура.

Эффект: после деплоя выполняется формализованный smoke-check, а не только ручная проверка «по ощущениям».

5.4. UART как режим по умолчанию

В последних изменениях `uart` закреплен как дефолтный MCU-транспорт с дополнительной подготовкой ОС (включая меры против `Permission denied` на `/dev/serial0`).

Эффект: более предсказуемый контур связи и снижение зависимости от USB-топологии.

6. Краткая динамика изменений (по ветке `dev`)

По истории коммитов на период 08.02.2026–12.02.2026 выполнены ключевые итерации:

- ? внедрение P0-контура MCU transport (`usb|uart`) и корректировка двухфазной миграции UART;
- ? усиление `verify` по готовности MCU;
- ? устранение silent-skip для обязательных шагов;
- ? доработка макросов печати, загрузки/выгрузки филамента и камерного цикла;
- ? выделение и доработка модуля бипера;
- ? внедрение темы KlipperScreen `treed-oled`;
- ? добавление инструмента валидации Klipper-конфигов.

Итог: проект перешел от «набора скриптов» к более инженерно формализованному пайплайну установки/проверки.

7. Текущее состояние готовности

На текущий момент можно считать, что:

- ? прошивочный контур RN12 под Klipper формализован и воспроизводим;
- ? профиль `rn12\_hbot\_v1` структурирован и пригоден для сопровождения;
- ? коммуникация MCU имеет контролируемую схему переключения и проверки;
- ? runtime-интеграции Moonraker/камера/макросы связаны в единый контур;
- ? в проекте внедрены базовые практики эксплуатационной устойчивости (идемпотентность, fail-fast, verify).

8. Ограничения текущей сводки и что добавить в НИР далее

Ограничения текущего документа:

- ? не включены полевые метрики (наработка, статистика печати, MTBF, частота ошибок по сменам);
- ? не приложена отдельная BOM-ведомость и расширенная схема питания/защиты;
- ? не включены результаты осциллографических/термальных измерений.

Рекомендуемые следующие шаги для НИР:

1. добавить эксплуатационные метрики (минимум 2–4 недели наблюдений);
2. оформить BOM и схему коммутации в отдельном приложении;
3. приложить протоколы тестов: холодный старт, reboot-циклы, стресс по длительной печати, отказ камеры/USB;
4. зафиксировать критерии приемки версии (release checklist).

Приложение: ключевые источники в репозитории

- ? `README.md`
- ? `docs/README.md`
- ? `docs/firstStart.md`
- ? `docs/rn\_v12\_to\_klipper.md`
- ? `docs/config-ownership.md`
- ? `loader/loader.sh`
- ? `loader/steps/README.md`
- ? `loader/steps/verify.sh`
- ? `loader/steps/klipper-profiles.sh`
- ? `klipper/printer.cfg`
- ? `klipper/profiles/rn12\_hbot\_v1/\*`
- ? `firmware/rn12/ROBIN\_NANO.bin`
- ? `moonraker/components/treed\_shell\_command.py`